

OCL: Invariantes

Una **invariante** es una restricción que debe ser cierta para un objeto durante todo su ciclo de vida

context <Typename>

inv [<inv name>]: -- expresión booleana que debe verificarse

- Una expresión **invariante** (*invariant*) debe ser cierta para cada instancia del **contexto** asociado en todo momento
- Una expresión **invariante** (*invariant*) debe ser cierta en todo estado consistente del sistema (mientras se está ejecutando una operación el sistema no se encuentra en un estado consistente)
- Todas las expresiones invariantes de OCL son del tipo Boolean

OCL: Pre y post-condiciones

- Una Precondición es una expresión booleana que debe ser cierta (true) en el momento en que la operación comienza su ejecución.
- Una Postcondición es una expresión booleana que debe ser cierta (true) en el momento en que la operación finaliza su ejecución.

context Typename::operationName(param1 : Type1, ...): ReturnType

pre [<pro name>]: -- expresión booleana que debe verificarse

post [<post name>]: -- expresión booleana que debe verificarse

self referencia un objeto en el cual la operación fue invocada

Los nombres de los parámetros (*param1*) se pueden utilizar en las expresiones OCL de las pre y/o postcondiciones

result palabra clave que indica el valor que retornará la operación. El **tipo de result** es definido por el tipo definido como retorno de la operación (*ReturnType*).

@pre Indica el valor de un atributo o asociación antes de la ejecución de una operación. **@pre** es sufixo del nombre de la propiedad.

oclIsNew Permite determinar si un objeto se creó durante la ejecución de una operación.

Reglas de Precedencia

@pre

'.' y '->'

operadores unarios 'not' y '-'

'*' y '/'

'+' y '-'

'if-then-else-endif'

'<', '>', '<=', '>='

'=', '<>'

'and', 'or' y 'xor'

'implies'

Paréntesis '(' y ')' permiten alterar el orden de precedencia

> precedencia

Operaciones Primitivas para el Tipo Boolean

Operación	Notación	Tipo del resultado
or	a or b	Boolean
and	a and b	Boolean
exclusive or	a xor b	Boolean
Iguales	a = b	Boolean
no iguales	a <> b	Boolean
implica	a implies b	Boolean
negación	not a	Boolean

context s:Socio

inv dniValido: s.dni > 0 and s.dni <= 99999999

Operaciones Primitivas para los Tipos Integer y Real

Operación	Notación	Tipo resultante
Iguales	a = b	Boolean
no iguales	a <> b	Boolean
menor	a < b	Boolean
mayor	a > b	Boolean
menor o igual	a <= b	Boolean
mayor o igual	a >= b	Boolean
suma	a + b	Integer o Real
resta	a - b	Integer o Real
multiplicación	a * b	Integer o Real
división	a / b	Real
módulo	a.mod(b)	Integer
división entera	a.div(b)	Integer
valor absoluto	a.abs()	Integer o Real
máximo de a y b	a.max(b)	Integer o Real
mínimo de a y b	a.min(b)	Integer o Real
redondeo	a.round()	Integer
base	a.floor()	Integer

Operaciones Primitivas para el Tipo String

Operación	Notación	Tipo del resultado
concatenación	string.concat(string)	String
longitud	string.size()	Integer
a minúscula	string.toLowerCase()	String
a mayúscula	string.toUpperCase()	String
substring	string.substring(int,int)	String
Iguales	string1 = string2	Boolean
no iguales	string1 <> string2	Boolean

Propiedades Definidas en Todos los Objetos

- La biblioteca estándar de OCL define el tipo **OclAny**.

OclAny es el supertipo de todos los tipos en un modelo, sus operaciones están definidas para todos los objetos OCL.

Expresión	Tipo del resultado
objeto = (objeto2 : OclAny)	Boolean
objeto <> (objeto2 : OclAny)	Boolean
objeto.oclIsUndefined()	Boolean
objeto.oclIsKindOf(tipo : OclType)	Boolean
objeto.oclIsTypeOf(tipo : OclType)	Boolean
objeto.oclIsNew()	Boolean
objeto.oclAsType(tipo : OclType)	Tipo
objeto.oclInState(nombre : StateName)	Boolean

Operaciones Comunes a Todas las Colecciones

Operación	Descripción
coleccion->count(objeto)	Cantidad de veces que ocurre <i>objeto</i> en <i>coleccion</i>
coleccion->excludes(objeto)	true si <i>objeto</i> no pertenece a <i>coleccion</i>
coleccion->excludesAll(coleccion2)	true si todos los elementos de <i>coleccion2</i> no están presentes en <i>coleccion</i>
coleccion->includes(objeto)	true si <i>objeto</i> está en <i>coleccion</i>
coleccion->includesAll(coleccion2)	true si todos los elementos de <i>coleccion2</i> están presentes en <i>coleccion</i>
coleccion->isEmpty()	true si <i>coleccion</i> no posee elementos
coleccion->notEmpty()	true si <i>coleccion</i> contiene uno o mas elementos
coleccion->size()	El número de elementos en <i>coleccion</i>
coleccion->sum()	La suma de todos los elementos de <i>coleccion</i> . Los elementos deben ser de un tipo que admite la suma

Operaciones según la Colección Set

Operación	Descripción (self es de tipo Set(T))
union(s: Set(T)) : Set(T)	unión de self y s
union(b: Bag(T)) : Bag(T)	Bolsa que contiene a todos los elementos de self y b
= (s: Set(T)) : Boolean	es true si self y s poseen los mismos elementos
intersection(s: Set(T)) : Set(T)	conjunto formado por los elementos que están en self y s
intersection(b: Bag(T)) : Set(T)	conjunto formado por los elementos que están en self y b
including(o: T) : Set(T)	conjunto formado por self + o
excluding(o: T) : Set(T)	conjunto formado por self sin el elemento o
symmetricDifference(s: Set(T)) : Set(T)	conjunto formado por los elementos que están en self o s, pero no en ambos
count(o: T) : Integer	número de ocurrencias de o en self (0 o 1)
flatten() : Set(T2)	si el tipo de self no es un tipo de colección, entonces el resultado es self. Sino, el resultado es el conjunto que contiene todos los elementos de self
asSet() : Set(T)	self
asOrderedSet() : OrderedSet(T)	conjunto ordenado que contiene todos los elementos de self (en un orden indefinido)
asSequence() : Sequence(T)	secuencia que contiene todos los elementos de self (en un orden indefinido)
asBag() : Bag(T)	bolsa que contiene todos los elementos de self

Operaciones según la Colección OrderedSet

Operación	Descripción (self es de tipo OrderedSet(T))
append(o: T) : OrderedSet(T)	conjunto ordenado que consiste de todos los elementos de self, seguido por o
prepend(o: T) : OrderedSet(T)	conjunto ordenado que consiste de todos los elementos de self, precedido por o
insertAt(i: Integer, o: T) : OrderedSet(T)	self con o insertado en la posición i
subOrderedSet(l: Integer, u: Integer) : OrderedSet(T)	subconjunto de self, contiene los elementos desde l hasta u inclusive
at(i: Integer) : T	i-ésimo elemento de self
indexOf(o: T) : Integer	la posición de o en la secuencia
first() : T	primer elemento en self
last() : T	último elemento en self

Operaciones según la Colección Bag

Operación	Descripción (self es de tipo Bag(T))
union(s: Set(T)) : Bag(T)	Bolsa que contiene a todos los elementos de self y s
union(b: Bag(T)) : Bag(T)	Bolsa que contiene a todos los elementos de self y b
= (b: Bag(T)) : Boolean	es true si self y b poseen los mismos elementos, el mismo número de veces
intersection(s: Set(T)) : Set(T)	conjunto formado por los elementos que están en self y s
intersection(b: Bag(T)) : Bag(T)	bolsa formada por los elementos que están en self y b (la cantidad mínima de cada uno de los elementos)
including(o: T) : Bag(T)	bolsa formada por self + o
excluding(o: T) : Bag(T)	bolsa formada por self sin ocurrencias del elemento o
count(o: T) : Integer	número de ocurrencias de o en self
flatten() : Bag(T2)	si el tipo de self no es un tipo de colección, entonces el resultado es self. Sino, el resultado es la bolsa que contiene todos los elementos de Self
asSet() : Set(T)	conjunto que contiene todos los elementos de self, eliminando duplicados
asOrderedSet() : OrderedSet(T)	conjunto ordenado que contiene todos los elementos de self, eliminando duplicados (en un orden indefinido)
asSequence() : Sequence(T)	secuencia que contiene todos los elementos de self (en un orden indefinido)
asBag() : Bag(T)	self

Operaciones según la Colección Sequence

Operación	Descripción (self es de tipo Sequence(T))
count(o: T) : Integer	número de ocurrencias de o en self
= (s: Sequence(T)) : Boolean	es true si self y s poseen los mismos elementos, en el mismo orden
union(s: Sequence(T)) : Sequence(T)	secuencia formada por todos los elementos de self, seguida por todos los elementos de s
flatten() : Sequence(T2)	si el tipo de self no es un tipo de colección, entonces el resultado es self. Sino, el resultado es la secuencia que contiene todos los elementos de self (el orden de los elementos es parcial)
append(o: T) : Sequence(T)	secuencia que consiste de todos los elementos de self, seguida por o
prepend(o: T) : Sequence(T)	secuencia que consiste de todos los elementos de self, precedida por o
insertAt(i: Integer, o: T) : Sequence(T)	self con o insertado en la posición i
subSequence(l: Integer, u: Integer) : Sequence(T)	subconjunto de self, contiene los elementos desde l hasta u inclusive

Operaciones según la Colección Sequence

Operación	Descripción (self es de tipo Sequence(T))
at(i: Integer): T	i-ésimo elemento de self
indexOf(o: T): Integer	la posición de o en la secuencia
first(): T	primer elemento en self
last(): T	último elemento en self
including(o: T): Sequence	secuencia conteniendo todos los elementos de self mas o como último elemento
excluding(o: T): Sequence	secuencia conteniendo todos los elementos de self excepto o
asSet(): Set(T)	conjunto que contiene todos los elementos de self, eliminando duplicados
asOrderedSet(): OrderedSet(T)	conjunto ordenado que contiene todos los elementos de self, en el mismo orden, eliminando duplicados
asSequence(): Sequence(T)	self
asBag(): Bag(T)	bolsa que contiene todos los elementos de self, incluso los duplicados

Operación select

- `select` especifica un subconjunto de una colección
`collection->select(boolean-expression)`
- El resultado es una colección que contiene todos los elementos de *collection* que satisfacen *boolean-expression*

`collection->select(v | boolean-expression-with-v)`

- La variable *v* se denomina *iterator*
- v* itera sobre los elementos de *collection* y la expresión *boolean-expression-with-v* es evaluada para cada valor de *v*

`collection->select(v : Type | boolean-expression-with-v)`

Operación collect

- Permite especificar que una colección se deriva de otra colección. Itera sobre la colección original, capturando en la nueva colección el resultado de la evaluación de la expresión.

`collection->collect(v : Type | expression-with-v)`
`collection->collect(v | expression-with-v)`
`collection->collect(expression)`

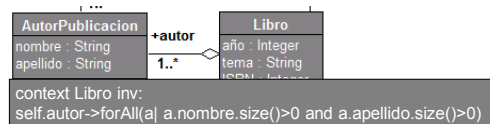
`collection->collect(propertyname)`

`collection.propertyname`

Operación forAll

- La operación *forAll* permite especificar una expresión que debe ser cierta para todos los objetos de una colección

`collection->forAll(v : Type | boolean-expression-with-v)`
`collection->forAll(v | boolean-expression-with-v)`
`collection->forAll(boolean-expression)`



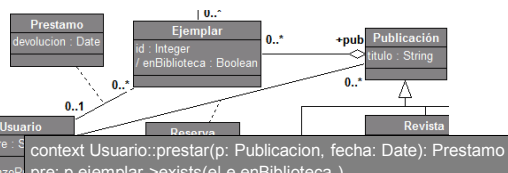
Es posible emplear más de un iterator

context Libro inv:
`Libro.allInstances()->forAll(I1, I2 | I1 <> I2 implies I1.ISBN <> I2.ISBN)`

Operación exists

- La operación *exists* permite especificar una expresión que debe ser cierta para al menos un objeto de la colección

`collection->exists(v : Type | boolean-expression-with-v)`
`collection->exists(v | boolean-expression-with-v)`
`collection->exists(boolean-expression)`



Operación iterate

- La operación *iterate* es mas general.

`collection->iterate(elem : Type; acc: Type= <expression> | expression-with-elem-and-acc)`

- La variable *elem* es el *iterator*.
- La variable *acc* es el *acumulador*, inicializado en el valor *<expression>*
- elem* itera sobre los elementos de *collection*
- La expresión *expression-with-elem-and-acc* se evalúa para cada *elem*
- Luego de cada evaluación de *expression-with-elem-and-acc*, su valor es asignado a *acc*.

`Set{1,2,3}->iterate(i:Integer; sum: Integer = 0 | sum + i)`